How to Pick the Model: A Framework for Budget-Aware Orchestration

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

Agent systems on verified tasks invite a deceptively simple question: which backbone model should be run? The strongest model is not always the best deployment choice, because verified progress must be bought under finite wall-clock, token, API, and evaluation budgets. We formulate this problem as deployment-aware model selection. A fixed router observes a pre-deployment information state and chooses a worker or harness action; the decision is scored by a checker-indexed deployment loss rather than by routing accuracy or final checker score alone. The key estimand is paired: on the same task instances, with the same router and action menu, does richer information change the selected action and reduce net deployment loss after its own measurement and context costs are charged? We instantiate the framework on a Karpathy AutoResearch-inspired CIFAR-10 edit–verify benchmark, where a prompt-only router chooses among heterogeneous workers before costly deployment runs. The protocol reports when checker-score rankings disagree with deployment-loss rankings, when pre-deployment information changes routing, and whether those changes pay for themselves. The resulting protocol evaluates not which agent is best in isolation, but when information is worth buying before deploying one.

1 Introduction

Practitioners building agent systems face multiple practical challenges: what modelwhich model (backbone) to use, how to route between models, when to spawn sub-agents, how to engineer context between them, and so on. All these possible choices often affect performance (quality of the solution), speed (time to solution), and the mere token costtoken/API cost. The question is how to pick between the different possible designschoose among the possible designs. The common way to answer this question is benchmark racing: take a benchmark assuch as SWE-bench [1] or MLE-bench [?][2] and pick the agent system with the best average end-to-end performance. An increasingly long line of work , based on this "recipe",based on this recipe is proposing better and better agentic system architectureagentic system architectures to achieve SOTA atstate-of-the-art performance on these tasks. This article investigates:This article investigates

whether there exist other ways to answer the question of what the most suited agent system is.
whether there are other ways to answer the question of which agent system is most suitable.

Indeed, (i) a large number of tasks for which such benchmark does not exist (as developing academic research [?]). And (ii) very often, the task presents instances of varied difficultythere are many tasks for which such benchmarks do not exist, such as developing academic research [3]. Moreover, tasks often contain instances of varied difficulty and the practitioner may want to understand whether, on the single instanceon a single instance, a lower-cost, faster model is sufficient; more tools should be given to the systemsthe system should be given more tools; or a more involved orchestration is required. As a running example for the rest of the paper, the question whether the user should use on their own task Haiku *vs* Sonnet *vs* Opus in Claude Code, or Claude Code *vs* Codex,the question of whether a user should use Haiku, Sonnet, or Opus in Claude Code, or Claude Code rather than Codex, for their own task is not fully addressed by benchmark racing.

The central issue is that one benchmark score may conflatesconflate mechanisms that imply different engineering decisions. A system can win because its worker takeshas lower per-step time or token-count. Because the selected worker istoken cost, or because the selected worker is more competent on a solver-relevant task mode. Analogously, imagine some sort of pre-deployment signala pre-deployment signal (problem definition, the context, or an initial testthe problem definition, context, or an initial test) reveals information about the instance whichthat can be used to better orchestrate or to choose what backbone to pickto choose a better backbone or orchestration. A system can win if it can orchestrate or choose the model optimally exploiting that information or notby exploiting that information when choosing a model or orchestration, or it can lose by failing to exploit it—as humans implicitly do when they decide to use lower or higher reasoning effort reasoning or different models on their tasks.

More practically, in this article *we study the deployment problem of choosing which model or harness to run on a verifiable task.* The objective is time to a required verified resulttime to reach a required verified result: among actions that may eventually succeed, the useful one is the one expected to reach the verifier threshold fastest, after charging any pre-deployment measurement used to make the choice. Achille and Soatto, indeed, recently showed that for verifiable tasks the time to solution, as in their Proper Time, is the performance measure to compare them. Indeed, Achille and Soatto [4] recently argued that time to solution, as formalized by their Proper Time, is a performance measure for comparing agents on verifiable tasks. Building on this intuition we operationalize the difference in time-to-solution of two possible designs. We show it mathematically decomposes in 4 components which isolates the mechanisms of choice mentioned above. Building on this intuition, we operationalize a certified log-effort surrogate gap between two possible designs. We show that it mathematically decomposes into four components that isolate the mechanisms of choice mentioned above. The experimental protocol then shows these can be estimated and lead to improvements in speed on Karpathy's AutoResearch tasks. The experimental protocol estimates the operational quantities that are directly measurable and tests, by paired selected-worker deployment, whether richer pre-deployment information improves net deployment loss after measurement costs are charged.

Research questions and contributions. We organize the paper around three questions, each paired with the contribution that answers it.

1. **Can the time-to-solutioncertified log-effort surrogate gap between agent designs be decomposed into meaningful terms?** Theorem 1 gives an exact four-term identity under a packed latent-mode assumption: cost, within-mode competence, routing information, and a predeclared mode-summary divergence additively explain the improvement in expected certified log-effort.
2. **Can the decomposition replacestructure or reduce end-to-end racing for model selection?** Algorithm 1 and the pilot concentration bound in Appendix A turn the identity into a structured pilot protocol whose evaluation cost depends on the number of models and task modes.
3. **When does a lower-cost model with retries beat a stronger model?** Theorem 2 gives the single-mode crossover rule, and the multi-mode accounting shows why the answer changes across task modes.

The experiments below instantiate these claims with checked trajectories and deployment accounting.

2 Problem setup and deployment estimands

We study the problem faced by a deployment system before launching an agentic run: given a task instance and a finite menu of possible workers or harnesses, which one should be run? The answer is not necessarily the model with the best final score. A deployment action must produce checked progress under finite time, token, API, and evaluation budgets.

Definition 1 (Budgeted externally checked task)**.** *A checked deployment taskbudgeted externally checked task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\text{dep}}, \mu).$$

Here $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is a artifact or solution spacesolution space, $V : \mathcal{X} \times \mathcal{Y} \to \mathcal{R}$ is an external checker or scoring procedure on instance–artifact pairs, B_{dep} is the full deployment budget and acceptance object, and μ is the deployment distribution over instances. The object B_{dep} may include a horizon, verifier budget, token/time/cost caps, and a thresholding rule that induces a binary success event from the checker score. For a realized instance $x \in \mathcal{X}$, an agent produces artifacts $y \in \mathcal{Y}$ that are scored by $V(x, y)$. An agent succeeds on $x \sim \mu$ if it produces an accepted y before exhausting B_{dep}.

The checker may be binary, as in unit tests or proof checking, or scalar, as in validation loss or reward. The key point is externality: progress is determined by the checker, not by the model's self-report.

Definition 2 (Workers, actions, and routers)**.** *Let $\mathcal{M} = \{M_1, \ldots, M_K\}$ be a finite menu of workers, where a worker is any deployable solver or fixed agent policy that can produce a candidate solution for verification. Let $\mathcal{A}$ be a finite menu of deployable actions. An action may be a single model, a fixed model harness, a retry policy, or a tool-using agent protocol. In the simplest setting, $\mathcal{A} = \mathcal{M}$; more generally, an action bundles a worker with fixed stopping, retry, tool-use, and carryover rules. The deployment loss is defined locally in this action definition. The deployment loss is defined in Section 4, where it is tied to first-passage cost, audit loss, and measurement loss. A routed design is a tuple*

$$d = (\mathcal{A}, R, \kappa),$$

where R is a router that selects an action after observing permitted information and κ is the declared scalar action-cost convention used in the log-effort diagnostic. When costs are logged as resource vectors, the protocol must fix the scalar reduction before evaluation. A router R observes a permitted pre-deployment information state $W(x)Z_j(x)$ and selects an action

$$a_j(x) = R(Z_j(x)) \in \mathcal{A}.$$

Equivalently, a signal condition induces the routing policy $\rho_j : x \mapsto a_j(x)$. The selected action then performs the deployment run from the original task state.

This separates routing from deployment. The router chooses what to run; the selected action performs the actual checked run. Unless explicitly declared otherwise, pre-deployment records are shown only to the router and are not passed to the selected worker.

Definition 3 (Task instances and task modes)**.** *Given $\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\text{dep}}, \mu)$, a task instance is a concrete realization $X = x \in \mathcal{X}$ of that same task. It is the measurable object supplied to the deployed system; the router and worker may access only the fields made available by the protocol. A task mode is a latent variable*

$$S = \sigma(X) \in \mathcal{S} = \{1, \ldots, m_S\}$$

induced by a measurable map $\sigma : \mathcal{X} \to \mathcal{S}$ that assigns each instance to one of finitely many solver-relevant variants of the task. It defines a prior $\pi_s = \mathbb{P}_\mu[S = s]$ and mode-conditional instance distributions $\mu_s = \mu(\cdot \mid S = s)$. Modes are therefore not different tasks and not worker labels. They are variants of the same task: finite subpopulations of instances that share solver-relevant structure and may favor different workers or interventions.

The mode variable is retained by the evaluator for accounting and diagnostic comparisons. It need not be observed by either the router or the worker. This lets the same theory cover expert-defined task-mode schemas and finite evaluator-defined modes. When an experiment uses one fixed executable prototype per mode, it estimates conditional panel quantities rather than the population quantities induced by μ.

Assumption 1 (Packed latent-mode family)**.** *Each instance $X \sim \mu$ is associated with a mode $S = \sigma(X)$ in a finite mode set $\mathcal{S}$, with prior π. Conditional on mode s, action a has success probability $p(a, s) > 0$ and scalar cost*

$$\kappa(a, s) = \mathbb{E}[C_\delta(a, X; \xi) \mid S = s] > 0$$

under the declared deployment protocol, where C_δ is the normalized cost accumulated to the first certified success or the horizon. At deployment time, the router observes a signal $Z = z$ and selects one action. The evaluator defines the mode distribution induced by that signal,

$$\pi_z(s) = \mathbb{P}[S = s \mid Z = z].$$

The router is not required to output π_z or any mode distribution. Instead, the theory uses a fixed evaluator-side mode-summary distribution $q_z \in \Delta(\mathcal{S})$, with $q_z(s) > 0$ whenever $\pi_z(s) > 0$. This summary is not a router output; it is a theory-facing diagnostic distribution specified before evaluation. Write $a_z = \rho(z)$ for the action selected after observing z. For an instance whose true mode is s, the certified effort surrogate is proportional to

$$T_{\rho,q}(s,z) \propto \frac{\kappa(a_z,s)}{p(a_z,s)\,q_z(s)}. \tag{1}$$

Assumption 1 is deliberately restrictive. Real task instances can vary in infinitely many ways, while the theory uses a finite state abstraction. The assumption says that, for model selection, instances can be packed into modes with stable action success probabilities. The purpose is to isolate the regime in which cost, competence, information, and the predeclared mode-summary divergence are algebraically separable. Residual and calibration diagnostics measure when this finite packing is too coarse.

Definition 4 (Cost-adjusted certified log-effort diagnostic objective)**.** *For a fixed routed policy $\rho : Z \mapsto a_Z$ and a fixed evaluator summary q satisfying Assumption 1, define*

$$\mathcal{J}(\rho,q) = \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}[\log\kappa(a_Z,S) - \log p(a_Z,S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z) + \mathrm{KL}(\pi_Z \| q_Z)]\,. \tag{2}$$

Lower $\mathcal{J}$ means lower expected certified log-effort under the fixed diagnostic convention. The quantity q is not optimized by the router; it is fixed by the evaluator-side diagnostic rule. When several signal conditions are compared, $\mathcal{J}$ is applied to the corresponding policy summaries; Appendix C writes this comparison explicitly for Z_j versus Z_0.

The action quantities in (2) are $\kappa(a_Z,s)$ and $p(a_Z,s)$: how costly the selected action is on mode s and how often it succeeds there. The router and information quantities are Z, π_z, and q_z: what the router is allowed to observe, what that signal reveals about the task mode, and the evaluator-side mode-summary distribution used in the KL term.

Remark 1 (Meaning of the effort surrogate)**.** *Equation* (1) *says that, on a true mode s after signal z, certified effort increases with action cost, decreases with action success probability on that mode, and increases when the evaluator-side mode summary gives too little diagnostic mass to that mode. The router is not literally routing jobs to modes; modes are evaluator-defined task variants. The quantity $q_z(s)$ is a diagnostic summary, not a deployment probability. The KL term below is therefore a mode-summary divergence under the chosen diagnostic convention. It should be interpreted as a router failure only if the experiment predefines a reproducible construction of q_z from router behavior.*

3 Performance accounting

For a fixed signal value z, the routing part of the objective is the cross-entropy between the signal-induced mode distribution and the fixed evaluator-side mode-summary distribution:

$$\mathrm{CE}(\pi_z,q_z) = -\sum_{s\in\mathcal{S}} \pi_z(s)\log q_z(s) = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z\|q_z). \tag{3}$$

The KL term is a divergence over finite task-mode distributions. It is not a token-level language-model divergence.

Theorem 1 (Four-term routed-policy decomposition)**.** *Under Assumption 1, compare a baseline policy ρ_0 that deploys fixed action a_0 without using Z and uses $q_0(\cdot \mid z) \equiv \pi$, with a deployed routed policy summarized by (ρ,q). The improvement in expected certified log-effort, $\Delta = \mathcal{J}(\rho_0,q_0) - \mathcal{J}(\rho,q)$, decomposes as*

$$\Delta = \underbrace{\mathbb{E}_Z\mathbb{E}_{S\sim\pi_Z}\left[\log\frac{\kappa(a_0,S)}{\kappa(a_Z,S)}\right]}_{\textit{cost}} + \underbrace{\mathbb{E}_Z\mathbb{E}_{S\sim\pi_Z}\left[\log\frac{p(a_Z,S)}{p(a_0,S)}\right]}_{\textit{competence}} + \underbrace{\mathrm{I}_\pi(S;Z)}_{\textit{routing information}} - \underbrace{\mathbb{E}_Z[\mathrm{KL}(\pi_Z\|q_Z)]}_{\textit{mode-summary divergence}}\,. \tag{4}$$

Proof sketch. Taking logs in (1) gives $\log T_{\rho,q}(s,z) = C + \log\kappa(a_z,s) - \log p(a_z,s) - \log q_z(s)$. Conditioning on $Z = z$ and averaging over $S \sim \pi_z$ converts $-\mathbb{E}\log q_z(S)$ into the cross-entropy in (3). Subtracting the prior-matched baseline cancels constants and uses $\mathrm{H}(\pi) - \mathbb{E}_Z\,\mathrm{H}(\pi_Z) = \mathrm{I}_\pi(S;Z)$. The full algebra is given in Appendix A. □

The four terms have different operational meanings. If the cost term is large and positive, a lower-cost action may be preferable even when it reaches success later in step count. If the competence term varies by mode, a global worker ranking is likely to hide comparative advantage. If information gain is high, the pre-deployment signal exposes solver-relevant structure. If the mode-summary divergence is high, the signal-induced mode distribution is poorly matched to the evaluator's predeclared mode summary. For two signal conditions, the identity compares the routed policies induced by those signals. A richer signal is useful only if its induced action change improves the cost–success tradeoff after the cost of obtaining the signal is charged.

Operational reading. The task mode S is not an oracle label for the best worker. The evaluator fixes a mode schema before evaluation, then estimates the mode-conditional quantities $p(a, s)$ and $\kappa(a, s)$. Routing information asks whether a signal makes those modes more identifiable. Routing shift asks whether the selected action changes when the signal is enriched. Mode-summary divergence is the KL term defined by the fixed evaluator-side mode summary q. Allocation regret, used later in the empirical protocol, is a separate action-frontier diagnostic rather than an estimator of this KL term. The final decision is not made by the information term alone; it is made by paired deployment gain on held-out instances after measurement and context costs are charged. Appendix B gives the full operationalization of these diagnostics.

3.1 When lower-cost retries win

Theorem 2 (Single-mode retry crossover)**.** *Fix a single homogeneous mode, success threshold δ, and horizon H. Let actions h and ℓ have one-attempt first-passage success probabilities $p_h = \mathbb{P}[\tau_\delta(h, X) \leq H]$ and $p_\ell = \mathbb{P}[\tau_\delta(\ell, X) \leq H]$, with expected first-passage costs κ_h and κ_ℓ. Assume $0 < p_\ell < p_h < 1$ and $\kappa_\ell < \kappa_h$. Assume retries of ℓ are conditionally i.i.d. fresh attempts, each with the same one-attempt horizon H and common Bernoulli success probability p_ℓ. The lower-cost retry depth that matches the strong action's one-attempt success probability is*

$$D_{\text{cross}} = \frac{\log(1 - p_h)}{\log(1 - p_\ell)}.$$

The corresponding integer fixed-depth retry block is cost-favorable whenever

$$\lceil D_{\text{cross}} \rceil \, \kappa_\ell < \kappa_h.$$

Remark 2 (Stopping after first retry success)**.** *If retries of the lower-cost action stop after the first success and each attempted retry has unconditional expected cost κ_ℓ, the expected cost at depth D is*

$$\kappa_\ell \sum_{d=1}^{D} (1 - p_\ell)^{d-1} = \kappa_\ell \frac{1 - (1 - p_\ell)^D}{p_\ell},$$

and the decision should also account for the residual failure probability under the deployment loss. Any difference in within-attempt time-to-success is already absorbed into κ_h and κ_ℓ.

The multi-mode case is where the four-term identity matters. A lower-cost worker can be rational on one task mode and irrational on another. A router is useful only when different actions win the cost–success tradeoff on different modes and the pre-deployment information helps identify which mode applies.

4 Decomposition-guided selection

The decomposition suggests a practical protocol. Rather than evaluate every complete design as an independent arm, the builder first estimates shared quantities: mode-conditional success, cost, predeclared signal diagnostics, and action-allocation diagnostics. Then a larger finite design menu can be scored analytically, with expensive deployment evaluations reserved for finalists and for paired validation.

The log-effort objective $\mathcal{J}$ is a diagnostic surrogate, not the deployment loss. For deployment validation we use a predeclared first-passage loss. Given a checked progress trace $G_1, \ldots, G_H$, define

$$\tau_\delta(a, x; \xi) = \inf\{t \leq H : G_t(a, x; \xi) \geq \delta\}, \qquad F_\delta(a, x; \xi) = \mathbf{1}\{\tau_\delta(a, x; \xi) = \infty\},$$

and

$$\mathrm{Occ}_{\delta,H}(a,x;\xi) = H^{-1}\sum_{t=1}^{H}\mathbf{1}\{G_t(a,x;\xi)\geq\delta\}.$$

Let $c_\delta(a,x;\xi)$ be the normalized worker-side resource vector accumulated to $\tau_\delta \wedge H$, and let

$$C_\delta(a,x;\xi) = \lambda^\top c_\delta(a,x;\xi)$$

be the scalar cost used in $\kappa(a,s)$. The primary loss is

$$\ell_{\mathrm{dep}}(a,x;\xi) = \alpha_{\mathrm{fail}}F_\delta(a,x;\xi) + C_\delta(a,x;\xi), \tag{5}$$

with $\alpha_{\mathrm{fail}} = 1$ and normalized resource weights λ fixed before evaluation. This is the loss used in $\mathcal{J}_R(j)$ and $\Delta_R(j)$ unless a different primary loss is explicitly declared. Because the benchmark records full trajectories to horizon H, we also report the full-horizon audit loss

$$\ell_{\mathrm{audit}}(a,x;\xi) = \ell_{\mathrm{dep}}(a,x;\xi) + \alpha_{\mathrm{occ}}\big(1-\mathrm{Occ}_{\delta,H}(a,x;\xi)\big) + \alpha_{\mathrm{qual}}\psi(G_H(a,x;\xi)), \tag{6}$$

where $\alpha_{\mathrm{occ}} = \alpha_{\mathrm{qual}} = 1$ and $\psi(u) = 1-u$ after clipping $G_H \in [0,1]$. The audit loss checks persistence and final quality; it does not change the first-passage cost convention in Theorem 1.

Algorithm 1 (Theory-to-deployment selection protocol).

Input: Task family, checker, finite task-mode schema $\mathcal{S}$, action menu $\mathcal{A}$, admissible progressive signals $\{Z_j\}$, pilot cell count n_{cell}, deployment repeats Q_{dep}, confirmation budget.

Output: Recommended signal-conditioned deployment policy, reported accounting terms, paired deployment gain, and calibration diagnostics.

1. *Freeze the modes and loss.* Declare $\mathcal{S}$, success threshold δ, horizon H, cost normalizers, deployment loss (5), measurement costs, and router output contract.
2. *Structured pilot.* For each action $a \in \mathcal{A}$ and mode $s \in \mathcal{S}$, estimate $\hat{p}(a,s)$, $\hat{\kappa}(a,s)$, first-hit curves, occupancy, and uncertainty intervals.
3. *Signal and router diagnostics.* For each progressive signal Z_j, report any predeclared signal-information diagnostic and log the action allocation induced by the router.
4. *Accounting score.* Compute the four log-effort terms from Theorem 1 only for quantities whose diagnostic estimators were predeclared; otherwise report allocation regret separately.
5. *Paired deployment validation.* On the same holdout instances, run the selected action under Z_0 and richer Z_j conditions, score repeated selected-worker trajectories by (5), and subtract measurement cost.
6. *Decision.* Choose the signal condition with the largest held-out $\widehat{\Delta}_R(j)$ subject to predeclared gates; use $\mathcal{J}$, frontier scores, and allocation regret as diagnostics, not as the final deployment objective.

4.1 Progressive signals and paired router value

Let $Z_j(X;\zeta_j)$ be the router-visible pre-deployment record under signal condition j, where ζ_j denotes measurement randomness. Let Z_0 be the static signal observed before choosing a deployment action. For any admissible richer condition j, the router observes Z_j, pays the corresponding measurement and router-context cost, and selects an action

$$a_R^j(X) = R(Z_j(X;\zeta_j);U_R) \in \mathcal{A}.$$

Here U_R denotes router-side randomness; for deterministic routers it is fixed and suppressed. Thus the experimental comparison is between routing policies ρ_0, ρ_j induced by different signals for the same orchestrator R, not between different orchestrators. The normalized measurement loss is

$$\ell_{\mathrm{meas}}(j,x;\zeta_j) = \lambda_{\mathrm{meas}}^\top d_j(x;\zeta_j),$$

where d_j contains measurement wall-clock, tokens, checker calls, context expansion, parsing, retry, and latency costs. Set $\ell_{\mathrm{meas}}(0,x) = 0$. The weights λ_{meas} use the same predeclared normalizers as deployment resource cost unless a separate router-cost scale is declared. In the main protocol, pre-deployment probes are outside the selected action's deployment run: they are charged through ℓ_{meas} and do not reduce the later horizon H or verifier budget B_V.

The selected-action deployment objective is

$$\mathcal{J}_R(j) := \mathbb{E}\left[\ell_{\mathrm{meas}}(j, X; \zeta_j) + \ell_{\mathrm{dep}}(a_R^j(X), X; \xi)\right]. \tag{7}$$

The expectation averages over task instances, measurement randomness, router randomness, and deployment randomness unless a conditional estimand is explicitly declared. The router-facing value of signal condition j is the paired gain

$$\Delta_R(j) := \mathcal{J}_R(0) - \mathcal{J}_R(j) = \mathbb{E}\Big[\ell_{\mathrm{dep}}(a_R^0(X), X) - \ell_{\mathrm{dep}}(a_R^j(X), X) - \ell_{\mathrm{meas}}(j, X)\Big]. \tag{8}$$

Positive $\Delta_R(j)$ means that the richer signal improves net deployment loss for the same router on the same task distribution. Allocation shift alone is insufficient:

$$\mathrm{Shift}_R(j) = \Pr[a_R^j(X) \neq a_R^0(X)]$$

measures decision relevance, not benefit.

Proposition 1 (Unbiased paired router-gain estimator)**.** *Assume task instances are sampled i.i.d. from μ, the orchestrator, action menu, and signal protocols are fixed before evaluation, measurement costs are recorded without bias, and deployment seeds are independent conditional on selected action and instance. The router is run once per instance and signal condition; the Q_{dep} deployment repeats estimate the conditional expected loss of the selected action. For condition j, estimate*

$$\widehat{\Delta}_R(j) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{Q_{\mathrm{dep}}}\sum_{q=1}^{Q_{\mathrm{dep}}}\ell_{\mathrm{dep}}(a_{R,i}^0, x_i; \xi_{i,q}^0) - \frac{1}{Q_{\mathrm{dep}}}\sum_{q=1}^{Q_{\mathrm{dep}}}\ell_{\mathrm{dep}}(a_{R,i}^j, x_i; \xi_{i,q}^j) - \widehat{\ell}_{\mathrm{meas}}(j, x_i)\right].$$

If the repeated-run averages are unbiased for the expected deployment losses of the selected actions, then $\mathbb{E}[\widehat{\Delta}_R(j)] = \Delta_R(j)$.

When the same worker is selected under Z_0 and Z_j, using common deployment seeds preserves unbiasedness and reduces paired variance. When counterfactual outcomes for all actions are logged on the same instance, hindsight action regret can be reported as a diagnostic, but it is not the primary estimand. If the study fixes a finite prototype panel rather than sampling i.i.d. instances from μ, the same estimator describes that panel and should not be interpreted as a population-level deployment estimand.

5 Operational experimental protocol

This section specifies the controlled protocol used to test the routed-policy decomposition. The main study is a fixed-prototype repeated-run panel: it fixes three AutoResearch task modes and uses repeated agentic trajectories to estimate, for the same starting executable artifact, how worker choice, signal exposure, and routing affect first-passage success and cost. Accordingly, the primary estimates are panel analogues of the population quantities in Section 3. Aggregate panel summaries use the predeclared empirical mode weighting $\hat{\pi}(s) = 1/3$ for $s \in \mathcal{S}$.

Task modes. The task is AutoResearch-style executable-artifact optimization on CIFAR-10. Each task instance contains an editable `train.py`, a read-only data/checker module `prepare.py`, and a verifier that executes `train.py` and reports task-level diagnostics for validation performance, compute cost, optimization progress, and model complexity. In this protocol, each task mode is represented by one fixed executable prototype of the same CIFAR-10 task. Concretely, a task mode is specified by the tuple

$$\chi_s = (A_s, D, \mathrm{split}, u, \eta_s, V, B_V, H, \delta),$$

where A_s is the starting model class and editable `train.py` template, D is CIFAR-10, split is the fixed train/validation split, u is the model-initialization seed, η_s denotes starting optimization and architecture hyperparameters, V is the checker, B_V is the checker budget, H is the agent horizon, and δ is the success threshold. The protocol uses three task modes:

$$\mathcal{S} = \{\texttt{mlp\_flat}, \texttt{cnn\_compact}, \texttt{resnet\_micro}\}.$$

Execution protocol. Appendix D reports the full fixed configuration read from the current `train.py` templates and the CIFAR-10 checker in Table 5. The paper-facing worker menu is

$$\mathcal{M}_{\text{main}} = \{\texttt{gpt\_5\_4\_mini},\ \texttt{gpt\_5\_3\_codex},\ \texttt{gpt\_5\_4}\}.$$

In the main experiment, each deployable action is one worker under the common fixed prompt and harness, so $\mathcal{A} = \mathcal{M}$. In the current snapshot, `gpt_5_3_codex` and `gpt_5_4` are the mature comparison pair. `gpt_5_4_mini` is being filled in as the lower-cost replacement for the unavailable `gpt_5_3_codex_spark` condition. Each worker uses the same prompt. For every task-mode–worker cell, we run

$$n_{\text{cell}} = 10$$

independent trajectories from the same fixed `train.py`; the repeated-run index changes only the agentic trajectory. The router is a fixed prompt-only orchestrator that chooses one worker from $\mathcal{M}$ before deployment. For the primary protocol it is deterministic after fixing model identifier, prompt, parser, temperature, and seed, so $B_R = 1$ router draw is logged per task-mode–signal pair. We evaluate a sequential signal protocol

$$Z_0,\ Z_1,\ Z_2,\ Z_3.$$

The signals are nested: Z_j contains all fields of Z_{j-1} plus the additional fields declared for condition j. Z_0 contains only the task name, checker objective, worker menu, and budget. Z_1 adds the initial `train.py` and a static workload summary: starting model class, parameter count, optimizer and budget settings, dataset/checker identifiers, and parse/runtime availability. Z_2 adds a low-cost checker probe of the unmodified artifact at a predeclared probe budget. Z_3 adds the full unmodified-artifact baseline under the deployment checker budget. Probe records contain validation loss, validation accuracy, training seconds, total seconds, optimizer steps, parameter count, parse/runtime status, and the probe budget. Probe runs are measurement actions for the router only: they are charged through ℓ_{meas} and do not consume the selected worker's later deployment horizon or checker budget. Let $L_{\text{base}}(x)$ be the validation loss of the unmodified starting artifact for instance x under the fixed deployment checker. This evaluator-only baseline is used to score G_t in all conditions; it becomes router-visible only in Z_3 and is then charged as measurement information. In the fixed-prototype protocol this baseline is constant within each task mode. At edit step t, a worker produces a candidate `train.py`, the checker returns loss L_t, and checked progress is

$$G_t = \text{clip}_{[0,1]}\left(\frac{L_{\text{base}}(x) - L_t}{|L_{\text{base}}(x)| + \varepsilon_L}\right).$$

We set $\varepsilon_L = 10^{-8}$ in the primary analysis. The primary success event is first passage above the relative-improvement threshold

$$\tau_\delta = \inf\{t \le H : G_t \ge \delta\}, \qquad \delta = 0.05.$$

The fixed-prototype competence estimator is $\hat{p}_{\chi_s}(a) = n_{\text{cell}}^{-1} \sum_{r=1}^{n_{\text{cell}}} \mathbf{1}\{\tau_{\delta,a,s,r} \le H\}$, the panel analogue of $p(a,s)$. The primary cost $\hat{\kappa}_{\chi_s}(a)$ is the sample mean worker wall-clock accumulated to $\tau_\delta \wedge H$, the panel analogue of $\kappa(a,s)$; token count is reported as a sensitivity analysis. The primary deployment loss is the first-passage loss in (5); threshold occupancy and final checked quality are reported through the audit loss (6). For log scores, we use the predeclared smoothed success estimate $\tilde{p}_{\chi_s}(a)$ to avoid infinite values in zero-success pilot cells.

Experiment 1: worker frontier. Run all workers on all three task modes without a router:

$$3 \text{ task modes} \times 3 \text{ workers} \times n_{\text{cell}} = 90$$

full trajectories. Estimate $\hat{p}_{\chi_s}(a)$, $\hat{\kappa}_{\chi_s}(a)$, mean first-hit step, occupancy, final quality, and uncertainty intervals. For each task mode, construct the empirical cost–success frontier

$$\widehat{\mathcal{F}}(s) = \left\{a \in \mathcal{A} : \log \hat{\kappa}_{\chi_s}(a) - \log \tilde{p}_{\chi_s}(a) \le \min_{a' \in \mathcal{A}} \left[\log \hat{\kappa}_{\chi_s}(a') - \log \tilde{p}_{\chi_s}(a')\right] + \epsilon_{\text{frontier}}\right\},$$

where $\epsilon_{\text{frontier}}$ is a predeclared uncertainty tolerance. This experiment is the worker-side calibration for Theorem 1. It estimates fixed-prototype analogues of $\kappa(a,s)$ and $p(a,s)$ for every candidate action before any router is allowed to choose. The same estimates can be plugged into Theorem 2, because the first-passage success probabilities and first-passage costs determine when repeated deployments of a lower-cost worker can match the one-attempt success probability of a stronger worker.

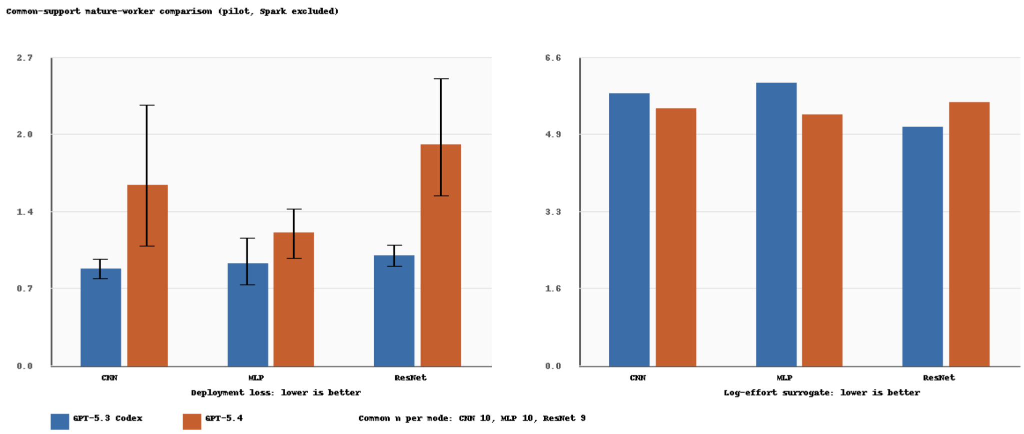

Figure 1: Mature-worker snapshot on common pilot support, excluding the rate-limited Spark condition. Left: deployment loss, where lower is better and intervals are the bootstrap intervals produced by the accounting script. Right: the certified log-effort surrogate used by the decomposition. The key qualitative result is a rank disagreement: `gpt_5_4` reaches the threshold faster and achieves higher final improvement, but the deployment-loss accounting currently favors `gpt_5_3_codex` in all three modes, and the log-effort surrogate favors different workers by mode.

Table 1: Pilot worker-frontier summary on common support. n_{common} is the number of paired trial indices retained for the two mature workers in that mode. The table reports first-hit success at $\delta = 0.05$, mean first-hit step among successful runs, mean threshold occupancy, mean final relative improvement, the log-effort surrogate, and deployment loss. Lower is better for the last two columns.

Mode	Worker	n_{common}	Success	Mean τ	Occupancy	Final improv.	Log-effort	Dep. loss
`cnn_compact`	`gpt_5_3_codex`	10	10/10	6.50	0.725	0.232	5.814	0.848
`cnn_compact`	`gpt_5_4`	10	10/10	3.30	0.885	0.373	5.503	1.580
`mlp_flat`	`gpt_5_3_codex`	10	9/10	8.11	0.580	0.171	6.045	0.890
`mlp_flat`	`gpt_5_4`	10	10/10	2.60	0.920	0.267	5.358	1.158
`resnet_micro`	`gpt_5_3_codex`	9	10/10	3.20	0.890	0.194	5.102	0.962
`resnet_micro`	`gpt_5_4`	9	9/9	1.67	0.967	0.282	5.626	1.934

Current worker-frontier results. Because the Spark condition hit rate limits, the main snapshot below compares the two mature workers only: `gpt_5_3_codex` and `gpt_5_4`. For each mode, we use the maximum common support available to both workers: ten pilot trajectories for `cnn_compact`, ten for `mlp_flat`, and nine for `resnet_micro`. The provisional `gpt_5_4_mini` pilot is intentionally not pooled into these estimates; it is reserved for the final lower-cost comparison once the cell reaches the same support.

This is the first empirical support for the paper's main claim. If one ranks workers by raw final improvement or by first-hit speed, `gpt_5_4` looks dominant. If one ranks by the deployment loss currently charged by the protocol, the recommendation changes. The experiment therefore already exhibits the failure mode of scalar leaderboard racing: different components of time-to-certified-solution point to different engineering choices.

Experiment 2: router-response audit. For each task mode and each signal Z_j, query the same orchestrator under the same worker menu. The router returns a structured decision record containing selected worker, confidence, short task diagnosis, evidence used, expected failure mode, and expected cost risk. The diagnostic question is whether progressive signals change the router's own decision record: selected-worker distribution, confidence, diagnosis consistency with the declared task mode, and cited evidence. This is a router-response diagnostic, not an estimator of $\mathrm{I}_\pi(S; Z)$. An information estimator would require a predeclared finite representation or predictive model for the signal records.

Current signal-audit results. The current router-facing diagnostic is not yet a deployment claim. It asks the narrower question required by the information term: whether admissible pre-deployment

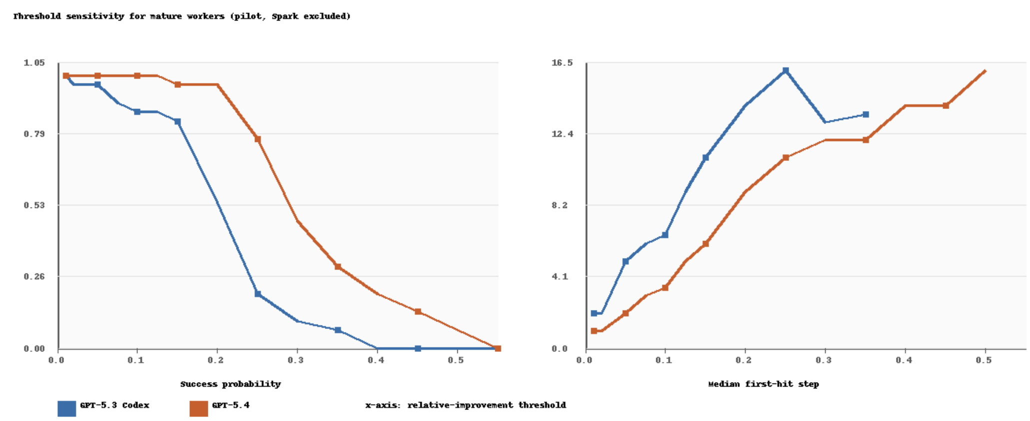

Figure 2: Threshold sensitivity for the same mature-worker comparison. The left panel shows first-passage success probability as the required relative-improvement threshold varies; the right panel shows the median first-hit step. At the paper's operating point $\delta = 0.05$, `gpt_5_4` has higher success probability and lower median first-hit time than `gpt_5_3_codex`. The fact that this speed advantage does not automatically imply lower deployment loss is the operational distinction the decomposition is meant to expose.

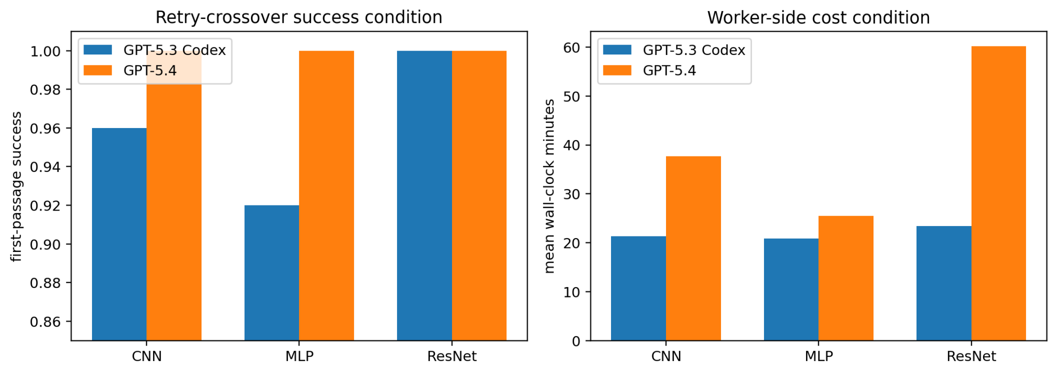

Figure 3: Applicability check for the retry-crossover theorem on the frozen two-worker confirmation split. The theorem requires a lower-cost action with lower one-attempt success probability and a stronger but more expensive action. The two mature workers do not cleanly enter this regime: `gpt_5_4` has both higher success and lower first-passage cost on `mlp_flat`, while both workers succeed on every frozen `resnet_micro` run and `gpt_5_3_codex` is cheaper there. The crossover result is therefore a design rule for adding a genuinely lower-cost worker, not a promoted empirical claim of the present two-worker menu.

records distinguish the task modes that have different empirical frontiers. On the current snapshot, budget-only records collapse the three modes to the same predicted class, while probe-containing records perfectly recover the predeclared mode labels.

This supports the premise that low-cost probes can reveal mode information, but it does not yet prove that the router uses that information correctly. The deployment-facing question is deferred to Experiments 3 and 4: whether the induced worker choice moves toward the empirical frontier and pays for the measurement cost.

Experiment 3: routing shift and allocation regret. Using the decisions from Experiment 2, compute the empirical panel shift rate

$$\widehat{\mathrm{Shift}}_R(j) = \sum_{s \in \mathcal{S}} \hat{\pi}(s) \mathbf{1}\{\rho_j(\chi_s) \neq \rho_0(\chi_s)\}$$

Table 2: Router-facing mode-diagnosis audit on the current signal records. Accuracy is reported only as a diagnostic for whether the signal exposes the finite task-mode structure; it is not the deployment objective.

Signal record	Records	Accuracy	Macro accuracy
Budget only	63	0.349	0.333
Probe only	63	1.000	1.000
Probe + budget	63	1.000	1.000
Leaky current-state control	63	1.000	1.000

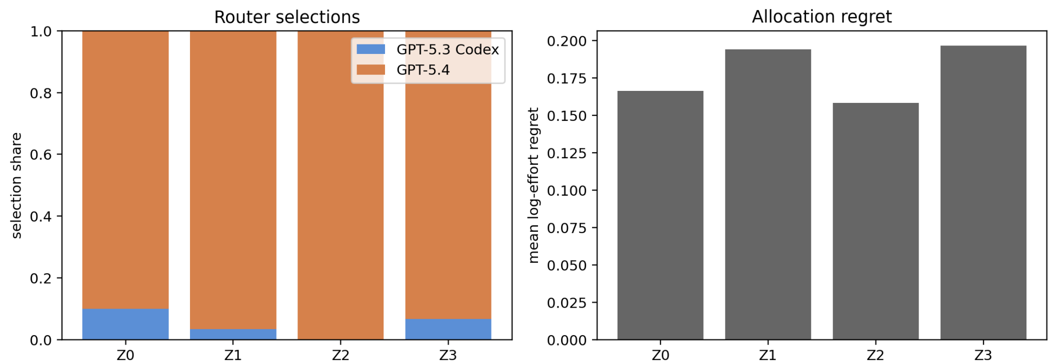

Figure 4: Final two-worker router-response audit. Left: selected-worker distribution under real signal records. Right: allocation regret against the frozen log-effort frontier. Richer records change the router only weakly and do not monotonically reduce regret, even though the signal audit shows that probes contain mode information.

for the deterministic primary router. Then use the frontier from Experiment 1 to score each selected action by allocation regret

$$\hat{r}_j(\chi_s) = \left[\log \hat{\kappa}_{\chi_s}(\rho_j(\chi_s)) - \log \tilde{p}_{\chi_s}(\rho_j(\chi_s))\right] - \min_{a \in \mathcal{A}} \left[\log \hat{\kappa}_{\chi_s}(a) - \log \tilde{p}_{\chi_s}(a)\right].$$

This is an operational action-frontier diagnostic, not an estimator of the KL mode-summary divergence $\mathbb{E}_Z \operatorname{KL}(\pi_Z \| q_Z)$ unless a separate construction of q_Z has been predeclared. When the same pilot runs define the frontier and score regret, regret is reported as an in-sample diagnostic; cross-fitted regret is used whenever enough pilot trajectories are available. Shift rate measures whether Z_j changes the routing policy, while allocation regret asks whether the changed policy moves toward the empirical mode-conditional frontier from Experiment 1. A signal can induce a router response and still increase allocation regret if it makes the orchestrator change worker in the wrong direction. Conversely, low regret indicates that the selected action moved closer to the empirical mode-conditional frontier rather than merely increasing decision variability.

Current routing-regret status. The final two-worker router audit contains 480 prompt-only decisions: 30 instances, four signal levels, and four control conditions. On the real signal records, the router remains strongly biased toward `gpt_5_4`: it selects `gpt_5_4` on 27/30 budget-only records, 29/30 static-workload records, 30/30 probe records, and 28/30 full-scout records. Mean log-effort regret does not decrease with richer information: it is 0.166 for Z_0, 0.194 for Z_1, 0.158 for Z_2, and 0.197 for Z_3. Thus the router-response audit separates information availability from useful allocation. The probe signals expose the predeclared task modes, but the prompt-only router mostly maps them to the same worker and does not reliably move toward the cost-adjusted frontier in Figure 5.

Experiment 4: paired selected-worker deployment gain. Report all $j \in \{1, 2, 3\}$ with multiplicity control as the primary analysis. If a single non-static signal is selected on a diagnostic split, evaluate it only on a separate holdout split. For each task mode and paired deployment index r, compare the worker selected under Z_0 with the worker selected under Z_j from the same original artifact and horizon. Pairing is by common prototype, common checker configuration, and matched deployment seed when the backend exposes seed control; otherwise intervals are stratified by task

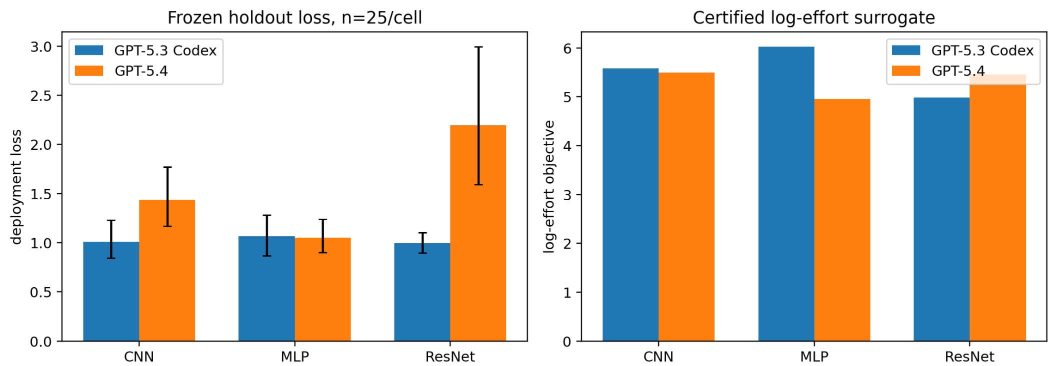

Figure 5: Frozen two-worker confirmation frontier with 25 holdout trajectories per task-mode–worker cell. The left panel reports deployment loss with bootstrap intervals; the right panel reports the certified log-effort surrogate. The confirmation split preserves the qualitative rank disagreement from the pilot: `gpt_5_4` has lower first-hit time, but deployment loss favors `gpt_5_3_codex` on `cnn_compact` and `resnet_micro`, while `mlp_flat` is nearly tied.

Table 3: Frozen confirmation summary for the two mature workers. Each cell uses the first 25 materialized holdout seeds under the frozen deterministic seed order. Deployment-loss intervals are bootstrap intervals over the 25 trajectories.

Mode	Worker	Success	Mean τ	Dep. loss	95% CI	Log-effort
`cnn_compact`	`gpt_5_3_codex`	24/25	5.83	1.011	[0.843, 1.231]	5.577
`cnn_compact`	`gpt_5_4`	25/25	2.96	1.438	[1.170, 1.770]	5.498
`mlp_flat`	`gpt_5_3_codex`	23/25	7.13	1.065	[0.867, 1.283]	6.028
`mlp_flat`	`gpt_5_4`	25/25	2.44	1.053	[0.901, 1.238]	4.955
`resnet_micro`	`gpt_5_3_codex`	25/25	2.36	0.997	[0.894, 1.105]	4.981
`resnet_micro`	`gpt_5_4`	25/25	1.84	2.193	[1.593, 2.996]	5.456

mode rather than treated as seed-paired. The selected-action confirmation uses $Q_{\text{dep}} = 10$ repeats per selected action and prototype unless a larger holdout budget is declared. Estimate

$$\widehat{\Delta}_R(j) = \widehat{\ell}_{\text{dep}}(\rho_0) - \widehat{\ell}_{\text{dep}}(\rho_j) - \widehat{\ell}_{\text{meas}}(j).$$

Report confidence intervals over the declared pairing unit, first-pass success changes, cost-to-success changes, gross loss reduction, measurement cost, and the subset where the signal changed the worker but increased loss. This experiment is the deployment validation of the theory-facing diagnostics. Proposition 1 gives the estimand and estimator for the same-orchestrator comparison, while (5) defines the loss used to decide whether a signal-conditioned policy is actually better after measurement cost is charged. Positive routing information and lower allocation regret are not sufficient by themselves; the routed-policy claim is supported only when the paired gain $\widehat{\Delta}_R(j)$ is positive on held-out trajectories.

Current validation status. For the two mature workers, we freeze the confirmation analysis at 25 runs per task-mode–worker cell:

$$3 \text{ task modes} \times 2 \text{ workers} \times 25 = 150$$

holdout trajectories. The frozen comparison uses `gpt_5_3_codex` and `gpt_5_4`; Spark is excluded because the endpoint hit rate limits, and `gpt_5_4_mini` is reserved for a later lower-cost extension once its cells reach comparable support. The paired-gain analysis therefore evaluates whether router-visible information improves deployment loss within this two-worker action menu.

Paired-gain result. The paired deployment validation is negative for the current prompt-only router. Against Z_0, richer real signals produce small shift rates and negative net gain after measurement cost: Z_1 shifts 13.3% of decisions with net gain -0.043 and bootstrap interval $[-0.100, -0.000]$; Z_2 shifts 10.0% with net gain -0.050, interval $[-0.107, -0.007]$; and Z_3 shifts 10.0% with net gain -0.057,

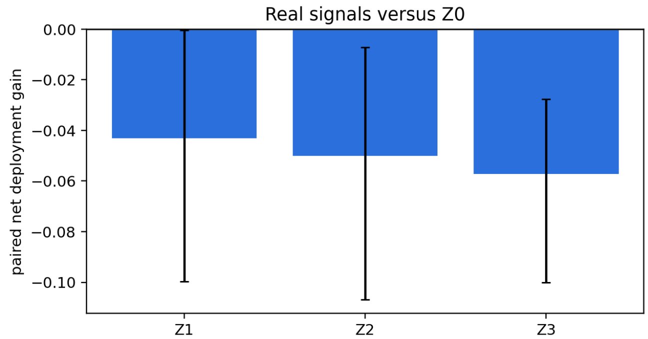

Figure 6: Paired deployment gain for real signal records relative to Z_0, using the frozen two-worker holdout frontier. All richer signal levels have negative net gain after measurement cost, so the current prompt-only router fails the deployment claim gate.

Table 4: Paired router validation on the final two-worker menu. Net gain is $\widehat{\ell}_{\mathrm{dep}}(\rho_0) - \widehat{\ell}_{\mathrm{dep}}(\rho_j) - \widehat{\ell}_{\mathrm{meas}}(j)$; positive is better.

Signal	Pairs	Shift	Gross gain	Meas. loss	Net gain
Z_1	30	0.133	-0.043	0.000	-0.043
Z_2	30	0.100	-0.043	0.007	-0.050
Z_3	30	0.100	-0.029	0.028	-0.057

interval $[-0.100, -0.028]$. The gross gains are already non-positive, and measurement cost makes the probe/scout conditions worse. This is the critical falsification result for the deployment-facing claim: informative signals are not sufficient; the deployed router must use them in a way that improves the selected action's checked cost–success tradeoff.

Experiment 5: negative controls. Repeat Experiments 2–4 with schema-preserving controls: shuffled probe records, probes from the wrong task mode, and synthetic noise records with the same field names and token length. The same router, prompt, worker menu, and scoring rule are used. These controls test whether router response, allocation regret, and paired deployment gain are specific to task-relevant signal content. They preserve the format, approximate context burden, and measurement/context cost of the corresponding real Z_j while breaking its relationship to the task mode and worker frontier. A credible deployment claim requires the real signal to reduce allocation regret and improve paired deployment gain more than these controls after measurement cost is charged.

Current negative-control status. The negative controls are also complete on the final two-worker menu. They do not show a clean separation between real task-relevant signals and corrupted signals. For example, wrong-mode Z_2 has a small positive mean net gain (0.017) with an interval crossing zero, while several shuffled or synthetic controls are negative with intervals similar to the real signals. The right interpretation is not that the controls discover a useful policy; it is that the prompt-only router has weak, noisy response to the records and the real signal does not dominate the controls. Accordingly, Experiment 5 blocks promotion of a strong routing-information claim. The paper should promote the worker-frontier and decomposition diagnostics, and report the router experiment as a falsification-style result for this particular router.

Interpretation through the two theorems. The four-term identity is empirically useful as a diagnostic accounting language. The worker-side terms show nontrivial cost and competence structure by mode; the signal audit shows that Z can expose task-mode information; the router audit then shows that information value is lost through allocation behavior. In the theorem's terms, positive routing information is not enough: allocation mismatch and worker-side costs can erase it, and the final deployment loss confirms that erasure. The retry-crossover theorem gives a complementary design rule, but the two mature workers do not instantiate its assumptions. There is no stable "cheap

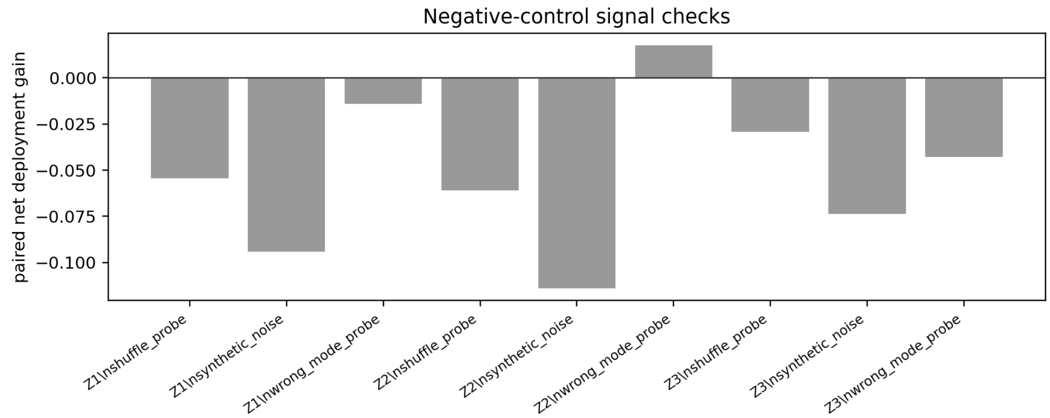

Figure 7: Negative-control paired gains for shuffled probes, wrong-mode probes, and synthetic noise records. The real records do not cleanly outperform the controls, which indicates that the current prompt-only router is not reliably exploiting task-relevant measurement content.

but weaker" action in the final two-worker menu: `gpt_5_4` is both stronger and cheaper to first passage on `mlp_flat`, while `gpt_5_3_codex` is cheaper on `resnet_micro` with no success-rate penalty. This is why `gpt_5_4_mini` is scientifically useful as a later extension: it would test the retry-crossover regime directly rather than merely adding another strong-worker comparison.

Claim promotion gates. We promote a routing recommendation only when the following are reported: mode-conditional competence with uncertainty, worker-side cost sensitivity, router shift, allocation regret, paired deployment gain with measurement cost charged, negative controls, calibration of $\tilde{p}$, and surrogate-versus-loss ranking agreement. If routing diagnostics improve but paired deployment gain does not, the conclusion is diagnostic rather than prescriptive. If paired deployment gain improves but surrogate-versus-loss agreement is poor, the protocol found a useful router but did not validate the packed-mode explanation.

6 Related work

The closest classical ancestor is algorithm selection. Since Rice's formulation, solver portfolios have treated performance as instance-dependent: SATzilla and AutoFolio learn to select solvers from instance features rather than report one globally best solver [5–8]. That literature also includes feature-computation costs, pre-solving, and probing. Our contribution is the specialization to externally checked LLM agents, where a router observes a controlled pre-deployment record, selects a worker or harness action, and is scored by paired deployment loss rather than by standalone routing accuracy. This differs from AutoML systems such as auto-sklearn, which select and tune learning pipelines offline for a dataset; here the unit is a deployable action for a concrete verifier-backed instance [9].

LLM routing and cascades provide the model-menu context. FrugalGPT frames model cascades as cost reduction across heterogeneous APIs, RouteLLM learns cost-quality routing from preference data, and recent compound-system routing work extends the idea to multi-module systems [10–14]. RouterBench and LLMRouterBench make this evaluation problem explicit at benchmark scale, with cost-, latency-, and quality-aware routing outcomes over large multi-model pools [15, 16]. These methods establish that heterogeneous model menus can create economic gains. The present paper asks how much checked, task-specific information is worth before committing to an expensive agentic deployment.

Checked agent benchmarks provide the task substrate. SWE-bench evaluates repository repair against tests, AutoResearch evaluates training-loop edits against validation feedback, and theorem-proving systems use analogous external validators [1, 17]. Most benchmarks ask how well a worker performs after it is launched. Our question is upstream: which worker should be launched, what should the router be allowed to observe, and how should the cost of that observation be charged?

Finally, repeated sampling and inference-time scaling show that lower-cost or smaller models can become competitive when verification is available [18–21]. The first-passage retry crossover in Theorem 2 is the simplest version of that tradeoff. The four-term identity embeds the same idea in a mode-aware routing and measurement account.

7 Discussion, future directions, and conclusion

There are two levels of claim. The theory-facing claim is an accounting identity under Assumption 1. It is exact, but only for the certified log-effort surrogate. The deployment-facing claim is empirical: the accounting should predict or explain useful action selection under a richer deployment loss. These two claims should not be collapsed. The current experiments support the central diagnostic claim: raw worker rankings, first-passage rankings, log-effort rankings, and deployment-loss rankings can disagree on the same checked task family. That disagreement is precisely the regime in which a budget-aware orchestration framework is needed. They do not support the stronger claim that the current prompt-only router improves deployment loss. Instead, the router validation is a useful negative result: task-relevant information is measurable, but a router that mostly defaults to the stronger model can still lose after measurement cost and worker-side costs are charged.

The packed-mode assumption is the main modeling limitation. Real agent trajectories are stateful, attempts are not independent, and task modes may be fuzzy. The paper therefore reports first-passage curves, occupancy, hazard diagnostics, cost sensitivity, and surrogate-versus-loss ranking agreement instead of assuming that the finite-mode surrogate is correct. If these diagnostics are unstable, the decomposition remains a useful diagnostic language but not a reliable selection rule.

The operational task-mode schema is also a limitation. If S is too coarse, mode-conditional competence is washed out and the router appears useless. If S is too fine, sample sizes per cell collapse and the signal–mode estimates become unstable. The present evaluation fixes the executable prototypes and mode schema to isolate agentic trajectory variance; task-instance and schema perturbations are left as robustness extensions rather than promoted experiments.

Finally, measurement value is router-specific. A baseline probe can contain information about the best action and still fail to improve deployment loss if the actual router does not use it well or if the measurement is too expensive. For this reason, the paired gain $\Delta_R(j)$ is the main deployment estimand, while mutual-information diagnostics, mode-summary divergence, and allocation regret help explain why that gain appears or fails to appear. This distinction is the main empirical lesson of the final two-worker study. The Z_2 and Z_3 records contain task-mode information, but the induced policies have negative paired gain. The decomposition therefore functions as a claim discipline: it prevents us from promoting information availability as deployment value.

Future directions. The immediate experimental extension is to finish the `gpt_5_4_mini` pilot and holdout cells, replacing the unavailable Spark worker with a stable lower-cost action. That would make the worker menu span a broader cost–capability frontier than the two-worker mature comparison reported here. It would also test the retry-crossover theorem in its intended regime, where a lower-cost worker may compensate for lower one-attempt success through repeated checked attempts. Beyond this benchmark, the same protocol should be tested on repository repair, theorem checking, and multi-file research-code editing, where the mode schema is less artificial and measurement costs are more heterogeneous.

Conclusion. Model choice inside an agent harness is not a scalar leaderboard problem. It is a measurable decomposition problem validated by paired deployment outcomes. The right worker depends on task state, checked competence, routing information, allocation quality, first-passage time, retry depth, and worker-side cost. Our CIFAR-10 edit–verify campaign provides the first empirical slice of that argument: the same two mature workers can trade off speed, final improvement, log-effort, and deployment loss in different ways. The two-worker routing analysis asks the deployment-facing question directly and answers it negatively for the current prompt-only router: pre-deployment information does not move allocation reliably enough to improve paired deployment loss after its measurement cost is charged. That negative result is not a failure of the framework; it is the framework doing its job, distinguishing worker-frontier structure from a validated routing recommendation.

References

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.

[3] Mahmoud Abdelmoneum, Pierfrancesco Beneventano, and Tomaso Poggio. PoggioAI/MSc: ML theory research with humans on the loop. Technical Report Technical Report v0, MIT, 2026.

[4] Alessandro Achille and Stefano Soatto. AI agents as universal task solvers: It's all about time. *Entropy*, 28(3):332, 2026.

[5] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.

[6] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

[7] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, Bernd Bischl, and Frank Hutter. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53:745–778, 2015.

[8] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2016.

[9] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[10] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[12] Lingjiao Chen et al. Optimizing model selection for compound AI systems. *arXiv preprint arXiv:2502.14815*, 2025.

[13] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.

[14] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research Blog, 2024.

[15] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

[16] Anonymous. LLMRouterBench: A massive benchmark and unified framework for LLM routing, 2026. Anonymous ACL submission.

[17] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. `https://github.com/karpathy/autoresearch`, 2026. GitHub repository.

[18] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[19] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

[20] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. In *International Conference on Learning Representations (ICLR)*, 2025.

[21] Jinghan Zhang, Kunhao Zheng, Yici Yan, Yuchen Ouyang, and Jun Zhu. Scaling LLM inference with optimized sample compute allocation. *arXiv preprint arXiv:2410.22480*, 2024.

A Proof details

A.1 Four-term identity

Under Assumption 1,

$$\log T_{\rho,q}(S,Z) = C + \log \kappa(a_Z,S) - \log p(a_Z,S) - \log q_Z(S).$$

Conditioning on $Z=z$,

$$\mathbb{E}[-\log q_z(S) \mid Z=z] = \sum_s \pi_z(s) \log \frac{1}{q_z(s)} = \mathrm{H}(\pi_z) + \mathrm{KL}(\pi_z \| q_z).$$

Therefore

$$\mathbb{E}[\log T_{\rho,q}] = C + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}[\log \kappa(a_Z,S) - \log p(a_Z,S)] + \mathbb{E}_Z[\mathrm{H}(\pi_Z)] + \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

For the prior-matched baseline ρ_0 that deploys fixed action a_0 and uses $q_0(\cdot \mid z) \equiv \pi$,

$$\mathbb{E}[\log T_{\rho_0,\pi}] = C + \mathbb{E}_{S\sim\pi}[\log \kappa(a_0,S) - \log p(a_0,S)] + \mathrm{H}(\pi).$$

Subtracting the deployed effort from the baseline effort yields

$$\Delta = \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{\kappa(a_0,S)}{\kappa(a_Z,S)}\right] + \mathbb{E}_Z \mathbb{E}_{S\sim\pi_Z}\left[\log \frac{p(a_Z,S)}{p(a_0,S)}\right] + \mathrm{H}(\pi) - \mathbb{E}_Z[\mathrm{H}(\pi_Z)] - \mathbb{E}_Z[\mathrm{KL}(\pi_Z \| q_Z)].$$

The entropy difference is $\mathrm{I}_\pi(S;Z)$, giving (4).

A.2 Pilot concentration

For binary success observations in an action-mode cell, Hoeffding gives

$$\mathbb{P}(|\hat{p}(a,s) - p(a,s)| > \alpha) \le 2\exp(-2n_0\alpha^2).$$

If all true probabilities are at least $p_{\min}$, then the log-probability error obeys $|\log \hat{p} - \log p| \le \alpha/p_{\min} + O(\alpha^2/p_{\min}^2)$ on the good event. A union bound over $|\mathcal{A}|\,|\mathcal{S}|$ cells yields the conservative pilot rule

$$n_0 = O\left(\frac{\log(|\mathcal{A}|\,|\mathcal{S}|/\delta_{\mathrm{conf}})}{\delta_{\mathrm{acc}}^2 p_{\min}^2}\right).$$

Wilson intervals and bootstrap intervals are reported in practice because pilot cell counts are small.

B Operational diagnostics for the four terms

The identity above becomes useful only after each term is tied to quantities logged by the deployment protocol. The checker-facing progress process is the common object used to operationalize competence, cost, information, and allocation diagnostics. Fix a horizon H and a task-normalized checked progress process $G_t(a,x) \in [0,1]$ for action a on instance x. In the main experiment, the router acts once before deployment: after observing Z, it selects one action, and that action is used for the whole trajectory. The success and cost diagnostics below are therefore trajectory-level quantities computed from the step-wise verifier trace. For lower-is-better checker losses, one admissible normalization is

$$G_t(a, x) = \mathrm{clip}_{[0,1]}\left(\frac{L_0(x) - L_t(a, x)}{|L_0(x)| + \varepsilon_L}\right),$$

where $L_0(x)$ is the starting-artifact loss and $L_t(a, x)$ is the checked loss convention at step t. Given a success threshold $\delta > 0$,

$$\tau_\delta(a, x) = \inf\{t \leq H : G_t(a, x) \geq \delta\}, \qquad F_\delta(a, x) = \mathbf{1}\{\tau_\delta(a, x) = \infty\}.$$

We also log persistence through

$$\mathrm{Occ}_{\delta,H}(a, x) = \frac{1}{H}\sum_{t=1}^{H} \mathbf{1}\{G_t(a, x) \geq \delta\}.$$

Competence. The action competence component $p(a, s)$ is the first-success probability of action a on task mode s:

$$p(a, s) = \mathbb{P}[\tau_\delta(a, X) \leq H \mid S = s].$$

Thus G_t is step-wise, but the policy competence term uses the probability that the worker selected by the router reaches the success threshold at least once before the horizon. Cumulative progress and persistence are not substituted for $p(a, s)$ in the identity. We log

$$\bar{G}_a(s) = \mathbb{E}\left[\frac{1}{H}\sum_{t=1}^{H} G_t(a, X) \mid S = s\right]$$

and $\mathrm{Occ}_{\delta,H}$ as secondary diagnostics, especially when first-hit success saturates or when partial progress is scientifically relevant.

Cost. The action cost component $\kappa(a, s)$ is kept separate from competence. In the main protocol it is the normalized cost of deploying action a on mode s for the same trajectory convention used to define $p(a, s)$. The routed-policy cost term averages this mode-conditional cost over the actions selected under Z. The primary convention is cost accumulated until the first certified success or the horizon, $\tau_\delta \wedge H$. We log

$$C_\delta(a, x; \xi) = \lambda^\top \big(\widetilde{T}^{\mathrm{worker}}_{\tau_\delta \wedge H}, \widetilde{C}^{\mathrm{tok}}_{\tau_\delta \wedge H}\big)(a, x; \xi),$$

and report worker wall-clock cost as primary, with token count as a sensitivity analysis. Checker runtime is logged for audit, but it is not part of $\kappa(a, s)$ unless the checker protocol or checker-call frequency differs across actions.

Routing information. The theorem term $\mathrm{I}(S; Z)$ measures how much a router-visible signal Z separates the predeclared task modes before deployment. This does not mean that the evaluator already knows the best worker for each instance. The evaluator knows only the candidate mode schema and can estimate whether Z_j makes that schema more identifiable:

$$\mathrm{I}(S; Z_j \mid Z_0) = \mathrm{H}(S \mid Z_0) - \mathrm{H}(S \mid Z_j).$$

The quantity is estimated only when a finite signal representation or calibrated predictive model for the signal records is predeclared. Otherwise the paper reports signal summaries and router-response diagnostics instead of a Shannon mutual-information estimate. It is a descriptive information diagnostic; its decision value is established only after action outcomes show that the modes predict different cost–success profiles. The paired router experiment then checks whether higher-information signals actually change the selected action and improve first-passage deployment outcomes.

Allocation-summary mismatch and allocation regret. The theorem term $\mathbb{E}_Z \,\mathrm{KL}(\pi_Z \| q_Z)$ is defined only after the evaluator predeclares a reproducible construction of the evaluator-side mode summary q_Z. Operationally, the router returns an action, not a mode label or distribution. When no such q_Z construction is frozen, the experiment reports allocation regret as a separate action-frontier diagnostic. After deployment runs, the evaluator estimates the mode-conditional frontier

$$a^\star(s) \in \arg\min_{a \in \mathcal{A}}\{\log \kappa(a, s) - \log p(a, s)\},$$

or the statistically indistinguishable set of frontier actions. For a selected action $a = \rho_j(x)$ and mode $s = \sigma(x)$, the corresponding allocation regret diagnostic is

$$r_j(x) = \big[\log \kappa(a, s) - \log p(a, s)\big] - \min_{a' \in \mathcal{A}} \big[\log \kappa(a', s) - \log p(a', s)\big].$$

This regret is computed retrospectively on a diagnostic split and validated on holdout instances; it is not an oracle input to the router. High information with high allocation regret means that the progressive signal exposed useful mode structure, but the induced action did not exploit the empirical cost–success frontier.

After these diagnostics are estimated, the experiment still reports the decision-facing first-passage deployment loss in (5) as final validation. The full-horizon audit loss in (6) is reported separately for persistence and final-quality checks. These losses are not replacements for the theorem; they test whether designs favored or explained by the diagnostics improve the practical deployment objective.

C Operational decomposition under progressive signals

The main four-term identity compares a deployed design to a prior-matched baseline. For the V3 router experiment, it is often useful to compare two progressive signal conditions for the same router. Assume a finite task mode $S \in \mathcal{S}$ and, for each signal condition j, the conditional mode distribution induced by the realized signal record

$$\pi_j(s \mid Z_j) = \mathbb{P}[S = s \mid Z_j].$$

The fixed orchestrator R induces the routing policy $a_R^j(Z_j) = R(Z_j)$. If an evaluator-side mode summary $q_R^j(\cdot \mid Z_j) \in \Delta(\mathcal{S})$ has been predeclared, define

$$\mathcal{E}_R^j(s, Z_j) = \frac{\kappa(a_R^j(Z_j), s)}{p(a_R^j(Z_j), s)\, q_R^j(s \mid Z_j)}.$$

The log-effort gain of signal condition Z_j over Z_0 is

$$\Delta_R^{\log}(j) := \mathbb{E}[\log \mathcal{E}_R^0(S, Z_0) - \log \mathcal{E}_R^j(S, Z_j)].$$

Expanding the cross-entropy terms gives

$$\Delta_R^{\log}(j) = C_R(j) + \Phi_R(j) + G(j) + M_R(j),$$

where

$$\begin{aligned}
C_R(j) &= \mathbb{E}\left[\log \frac{\kappa(a_R^0(Z_0), S)}{\kappa(a_R^j(Z_j), S)}\right], \\
\Phi_R(j) &= \mathbb{E}\left[\log \frac{p(a_R^j(Z_j), S)}{p(a_R^0(Z_0), S)}\right], \\
G(j) &= \mathrm{H}(S \mid Z_0) - \mathrm{H}(S \mid Z_j), \\
M_R(j) &= \mathbb{E}\Big[\mathrm{KL}(\pi_0(\cdot \mid Z_0) \| q_R^0(\cdot \mid Z_0)) - \mathrm{KL}(\pi_j(\cdot \mid Z_j) \| q_R^j(\cdot \mid Z_j))\Big].
\end{aligned}$$

When Z_j refines Z_0, $G(j) = \mathrm{I}(S; Z_j \mid Z_0)$. This signal-mode decomposition is theory-facing and excludes measurement cost unless such a term is added explicitly. It is a diagnostic companion to the deployment-gain estimand (8), not a replacement for it.

D Operational estimator details

Trajectory record. Each run stores the worker alias, resolved model identifier, API date, split, task family, task mode, seed, maximum horizon H, signal condition, prompt hash, router model identifier, router temperature/top-p, router seed policy, parser version, router output, selected worker, and cost convention. Each step stores the prompt-visible state, proposed `train.py` artifact, parse status, checker result, validation loss, validation accuracy, relative improvement, incremental worker wall time, token usage, and checker runtime for audit. Parser failures are logged as unsuccessful routing or deployment records under the predeclared parser-failure policy.

Table 5: Frozen operational configuration for the main protocol. All values are fixed within a task mode; repeated runs vary only the agentic trajectory.

Component	Fixed value
Dataset and split	CIFAR-10, 50,000 training examples and 10,000 validation examples; balanced subset, label noise 0.0, imbalance ratio 1.0; the train/validation split is fixed in the main protocol
Data seed	Common fixed data/subset seed; alternative split seeds are reported only as robustness variants
`train.py` initialization seed	`SEED=42`; used for Python, NumPy, PyTorch, CUDA seeds, deterministic PyTorch mode, and the training-loader generator; alternative initialization seeds are robustness variants
Training transforms	random crop with padding 4, random horizontal flip, CIFAR-10 normalization; validation uses normalization only
Training budget per checker call	fixed-step mode with `A_MAX_STEPS=256`; time budget `120s` is a fallback, not the primary scoring convention
Common hyperparameters	`BATCH_SIZE=64`, `NUM_WORKERS=0`, Adam, learning rate $5 \cdot 10^{-4}$, weight decay 10^{-4}, betas $(0.9, 0.999)$, no LR schedule
Task mode	`mlp_flat`, `cnn_compact`, or `resnet_micro`
Shared template parameters	depth 2, base channels 12, channel multiplier 2, batch norm enabled, dropout 0.0, and hidden width 48
Agent horizon	$H = 20$ edit–verify steps; trajectories continue to H after first success to measure persistence and final quality
Progress normalization	$\varepsilon_L = 10^{-8}$ in the denominator of G_t
Primary loss weights	$\alpha_{\mathrm{fail}} = 1$; normalized worker wall-clock is the primary scalar C_δ; token cost is a sensitivity analysis
Audit loss weights	$\alpha_{\mathrm{occ}} = \alpha_{\mathrm{qual}} = 1, \psi(u) = 1 - u$
Measurement cost	pre-deployment probes are charged through ℓ_{meas} using the same wall-clock normalizer as C_δ; context-token cost is reported as sensitivity
Router configuration	deterministic prompt-only router with fixed model identifier, prompt hash, parser, temperature, top-p, and seed policy recorded before evaluation

Success and time-to-verification. For trajectory i, τ_i is the first step at which the relative improvement threshold is crossed; censored failures have $\tau_i = \infty$. For each action-mode cell,

$$\hat{F}_a(s,h) = \frac{1}{n_{a,s}} \sum_i \mathbf{1}\{\tau_i \le h\}, \qquad \hat{S}_a(s,h) = 1 - \hat{F}_a(s,h).$$

The empirical hazard is

$$\hat{\lambda}_a(s,h) = \frac{\sum_i \mathbf{1}\{\tau_i = h\}}{\sum_i \mathbf{1}\{\tau_i \ge h\}}.$$

The raw first-success estimate is $\hat{p}(a,s) = \hat{F}_a(s,H)$. For log-effort scores, the protocol uses the predeclared smoothed estimate

$$\tilde{p}(a,s) = \frac{k_{a,s} + 1/2}{n_{a,s} + 1},$$

where $k_{a,s}$ is the number of first successes in the cell. This prevents zero-success pilot cells from making log scores infinite; raw success counts are still reported.

Cost. The primary cost is cumulative worker wall-clock time to $\tau \wedge H$. Secondary worker-side cost is cumulative token count. Checker runtime is logged as a protocol audit field and excluded from action-cost comparisons unless the checker protocol differs across actions. All plots that support model recommendations must be reproducible under at least worker wall-clock and token-count conventions. When a worker fails before producing a parseable artifact, its cost is still counted and the step is marked unsuccessful. The scalar estimator matched to Assumption 1 is the cell mean

$$\hat{\kappa}(a,s) = \frac{1}{n_{a,s}} \sum_{i:S_i = s} C_{\delta,i}(a),$$

or its fixed-prototype analogue $\hat{\kappa}_{\chi_s}(a)$ in Section 5.

Information, allocation summaries, and regret. For each progressive signal Z_j, a Shannon information estimate is reported only if the protocol predeclares a finite signal representation or calibrated predictive model for estimating $\pi_j(\cdot \mid Z_j)$ on held-out instances. Otherwise the report uses qualitative signal summaries and router-response diagnostics without calling them estimates of $\mathrm{I}(S; Z_j)$. The KL mode-summary divergence term is reported only if the evaluator also freezes a reproducible construction of $\hat{q}_R^j(\cdot \mid Z_{j,i})$ before seeing evaluation outcomes. In the main fixed-prototype protocol, the primary operational diagnostic is allocation regret,

$$\hat{r}_j(x_i) = \big[\log \hat{\kappa}(a_{j,i}, s_i) - \log \tilde{p}(a_{j,i}, s_i)\big] - \min_{a \in \mathcal{A}} \big[\log \hat{\kappa}(a, s_i) - \log \tilde{p}(a, s_i)\big],$$

where $a_{j,i} = R(Z_j(x_i))$ and $s_i = \sigma(x_i)$.

Uncertainty. Success probabilities use Wilson intervals. Aggregated objectives, paired gains, and calibration diagnostics use bootstrap or randomization tests at the sampling unit declared for the study. If the main protocol remains a fixed-prototype repeated-run panel, paired randomization tests over repeated trajectories are descriptive for that panel and should not be interpreted as population-level inference. If independent task instances are added, uncertainty is reported at the task-instance level.

E Additional diagnostic plots

This appendix collects auxiliary diagnostics generated from the experimental logs. Some plots include exploratory cells outside the promoted two-worker comparison; the main text uses the Spark-free mature-worker figures in Figures 1 and 2 for the primary comparison.

E.1 Pilot threshold and trajectory diagnostics

E.2 Pilot cost diagnostics

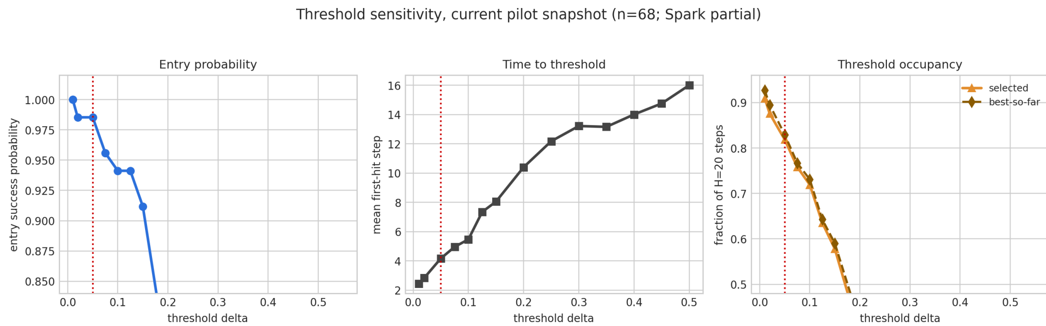

Figure 8: Zoomed pilot threshold sensitivity. The red line marks the operating threshold $\delta = 0.05$. At this threshold, entry success is near saturated while mean first-hit time remains low, which makes $\delta = 0.05$ a pragmatic operating point for studying time-to-certified-solution. The sharp drop at larger thresholds shows why the exact threshold is not an innocuous plotting choice.

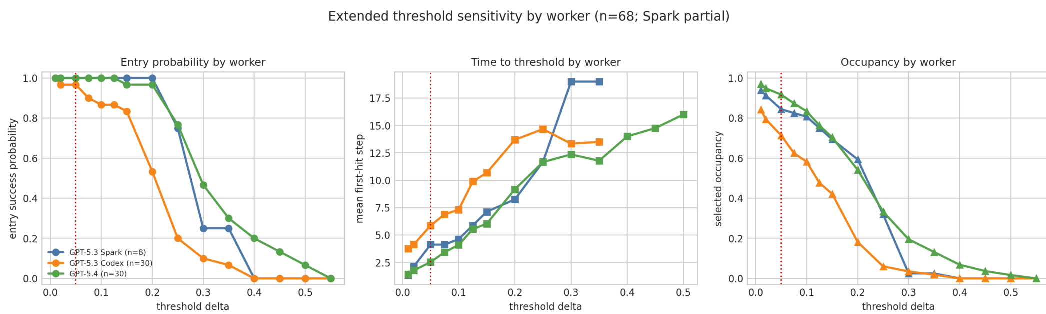

Figure 9: Threshold sensitivity by worker on the exploratory pilot snapshot. The promoted paper comparison excludes the rate-limited Spark condition, but the diagnostic is useful because it shows the worker-dependent shape of the success frontier. `gpt_5_4` remains stronger at higher thresholds, while `gpt_5_3_codex` degrades earlier; this supports the competence term in Theorem 1.

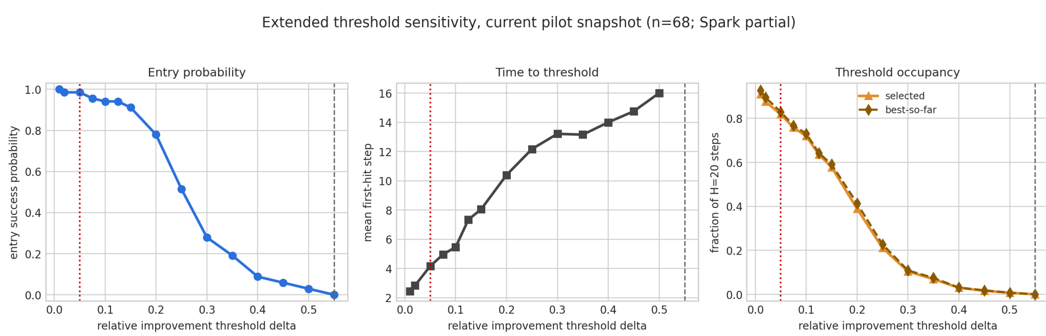

Figure 10: Extended threshold sensitivity. The full threshold range shows that high-threshold success is rare even when $\delta = 0.05$ is nearly saturated. This is why the paper reports first-passage curves, occupancy, and final quality rather than relying on a single binary success rate.

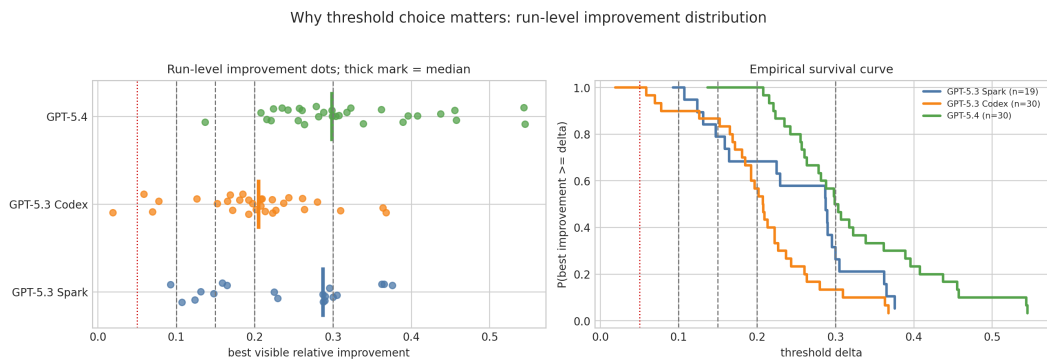

Figure 11: Run-level improvement distribution. Dots show final best-visible relative improvement and thick ticks mark worker medians. The survival plot makes the threshold choice explicit: near $\delta = 0.05$, most runs succeed, but the ordering and separation change at higher thresholds. This diagnostic supports the paper's claim that final quality, first-passage success, and deployment loss are distinct empirical objects.

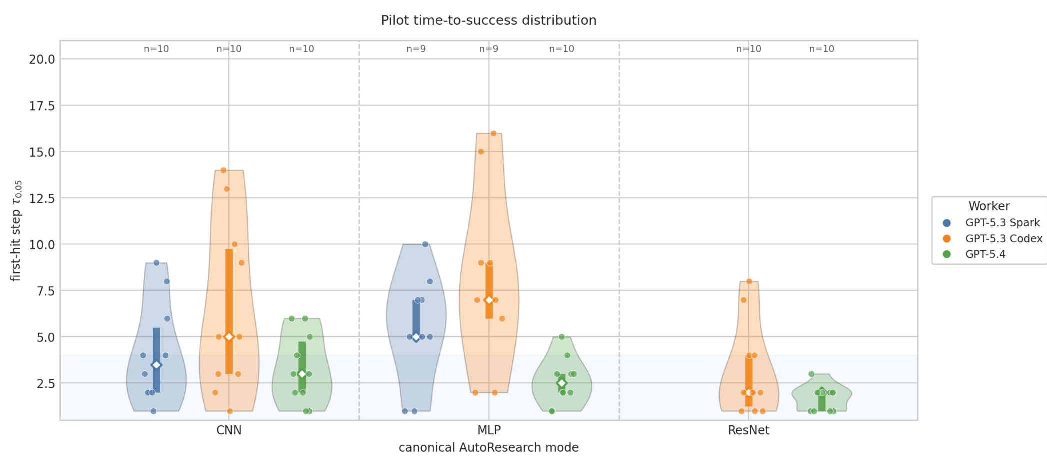

Figure 12: Pilot time-to-success distribution by task mode and worker. The violin plots expose mode-conditional heterogeneity: `gpt_5_4` typically hits the threshold quickly, while `gpt_5_3_codex` has broader first-hit times, especially on MLP. This is the empirical object behind the cost and competence terms of the four-term decomposition.

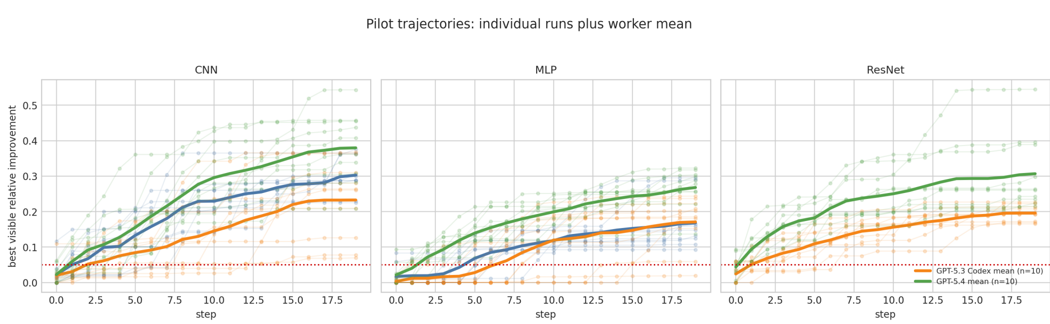

Figure 13: Best-so-far improvement trajectories. The mean trajectories show that `gpt_5_4` improves earlier and reaches higher final relative improvement in the pilot, but the holdout deployment accounting later shows that this does not automatically imply lower deployment loss after cost is charged. This contrast is the main empirical reason to separate worker competence from deployment value.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's scope: verifiable agentic systems, a packed latent-mode assumption, decomposition-guided selection, and a controlled fixed-prototype experimental protocol. The discussion section separates theory-facing and deployment-facing claims.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 discusses the packed-mode assumption, small live-agent sample sizes, finite design menus, oracle mode labels, and current cost-measurement limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Assumption 1 states the assumptions for the main theoretical result, Theorem 1 includes a proof sketch, and Appendix A gives the algebraic proof and pilot concentration details.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, horizons, metrics, and evaluation protocol; Appendix D specifies the required logs, estimators, and run-metadata fields.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The paper is written against an existing repository and includes reproducibility commands, but an anonymized public code/data release is not included in the current submission package.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, primary estimators, horizons, and evaluation design. Additional implementation details are listed in Appendix D.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Section 5 and Appendix D specify Wilson intervals, bootstrap intervals, survival curves, calibration diagnostics, and held-out evaluation for the reported protocol.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The paper reports wall-clock costs and run counts but does not provide a complete compute-resource table for every experiment.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work studies verifiable benchmark tasks and does not involve human subjects, sensitive personal data, or released high-risk models. The submission preserves anonymity and follows NeurIPS code and data policies.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 7 notes positive uses in cost-aware and transparent agent deployment as well as limitations and failure modes. The work is methodological and does not introduce a new generative model or dataset of people.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release a pre-trained model, image generator, scraped dataset, or other asset with high misuse risk. It evaluates agent harnesses on synthetic or generated benchmark tasks.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The related-work section cites existing papers and systems used for positioning, and the experimental section describes the benchmark surfaces. Released code and benchmark assets include explicit license details.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The work introduces a code harness and analysis artifacts as research assets, and Appendix D documents the required logging and configuration fields.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects, so IRB or equivalent approval is not applicable.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are central to the evaluated agent designs. Section 5 describes the three-worker menu, fixed prompt-only router, sequential signal protocol, model-identifier logging, and routing-output requirements.

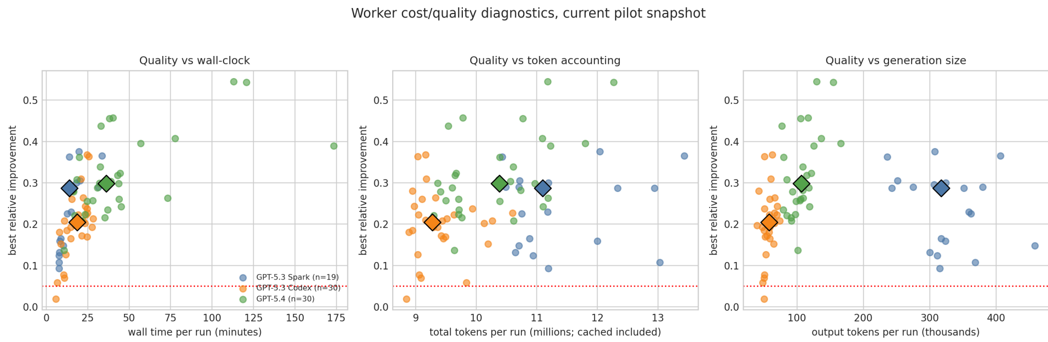

Figure 14: Worker cost–quality diagnostics. Each point is a pilot run, and diamonds mark worker medians. The plots show that better final improvement is accompanied by substantial variation in wall-clock time, total token accounting, and generated-token volume. This supports the paper's budget-aware framing: worker choice cannot be reduced to final improvement alone.

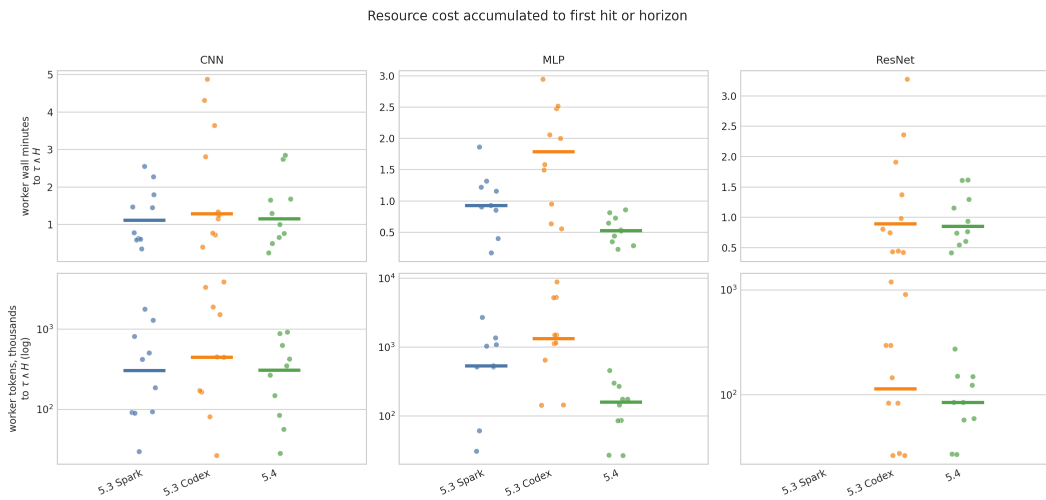

Figure 15: Resource cost accumulated to first hit or horizon. The top row reports wall-clock minutes and the bottom row reports token accounting on a log scale. Mode-conditional cost spreads are large, which is why the retry-crossover theorem is treated as a cost-sensitive design rule rather than a generic endorsement of weaker workers.

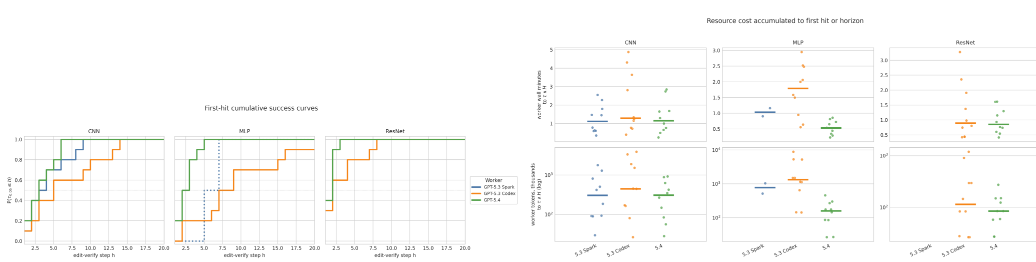

Figure 16: First-passage diagnostics. Left: first-hit CDFs by mode. Right: cost-to-threshold by mode and worker. These are the direct empirical objects behind the retry and time-to-certified-solution interpretation.

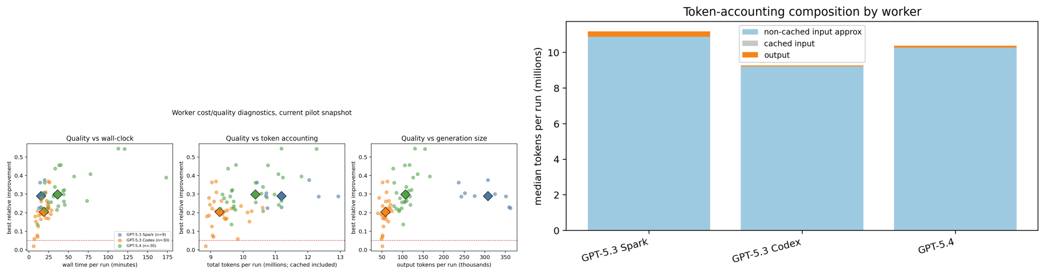

Figure 17: Worker-side cost and accounting diagnostics. These plots check whether conclusions are driven by wall-clock cost, token composition, or quality differences.

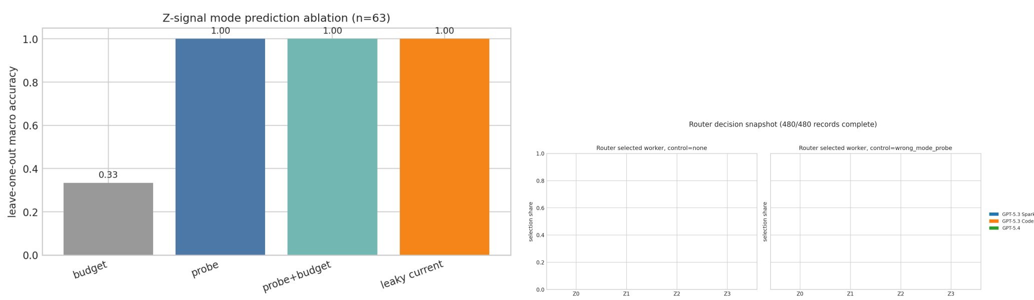

Figure 18: Router-facing diagnostics. The signal ablation tests whether progressively richer records expose task-mode structure; the router-selection snapshot checks whether the prompt-only router changes action distribution under those records.

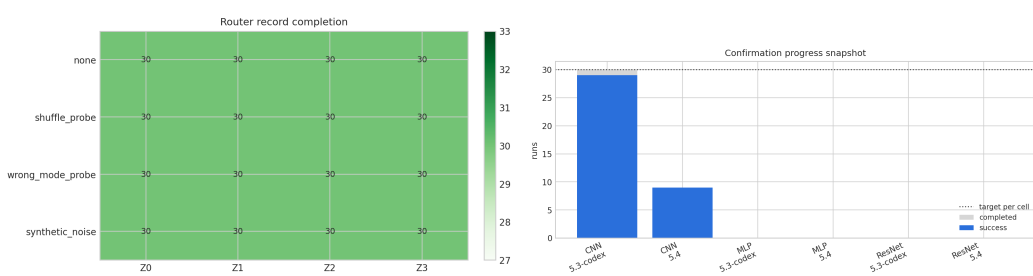

Figure 19: Run-coverage diagnostics for router records and holdout confirmation runs. These plots document which analyses are part of the promoted two-worker comparison and which are exploratory.